

A new way to monitor your vehicle, OBD-2GO!

CS 362 – Team 16

1. Team info:

Owen Koster,
Peyton DuPont
Tiernan Flanagan-Caldwell
Silas Jones

The main method of communication is Discord

2. Product description

Abstract:

A notorious source of headache for drivers has always been the dreaded check engine light. When it comes on, it leaves you with the anxiety of knowing something might be terribly wrong with your vehicle – Or it could be nothing. The OBD-2GO aims to make this system more transparent so drivers don't have to worry about what might be wrong with their vehicle. The OBD-2GO will be a system embedded into your vehicle that will actively monitor and warn you of issues with your vehicle in a more detailed manner than a simple check engine light. It will also enable you to monitor and record vehicle parameters while driving, so you can identify signs of failure before the problem becomes more expensive to resolve.

Goal:

Our goal is to develop software that provides users with more information than a simple check engine light. This will be done by displaying Diagnostic Trouble Codes (DTCs) that contain information about the failure detected by the vehicle. These are generated automatically by the car's computer and can be read by accessing the OBD2 port. DTCs are only half the story; it is also beneficial for both users and mechanics to see the vehicle's state before the check engine light is triggered, the state when the light is triggered, and the state afterward to properly diagnose and repair the root cause of the failure. We will help solve this problem by logging vehicles' live sensor data and storing it so the user or mechanic can home in on the root cause and properly repair it.

Current practices:

Currently, to access the DTCs and vehicle data, you need an OBD2 Scanner, which many people do not own or carry in their vehicles. These tools enable the vehicle owner or mechanic to read the detected failures and view live sensor data to diagnose the cause of the check engine light. For the most part, simple OBD2 scanners cost \$20-\$100, but the accompanying software is limited in capability and not designed to be a permanent feature in a vehicle.

Scanners that display live data intuitively cost \$500+ each. Almost all big box parts stores (Autozone, Napa, etc.) offer free code reading, where they will tell you the codes that caused the check engine light, but this means you have to stop what you are doing and physically go to a parts store and get the codes read which is a hassle if you are running late for work and worried about if you will damage the vehicle by continuing to drive it another 20-50 miles to work or school. Overall, there is no reliable way to see the cause of the check engine light while driving and determine whether it is safe to continue, and we aim to change that.

Novelty:

Our project is unique because we plan to provide realtime DTC detection and data logging while the vehicle is operating, instead of just taking care of problems when they appear. Our software would eventually be implemented as an embedded system inside a vehicle, and designed to be a constant fixture inside of a vehicle. As of right now we plan on first developing the software to work as a desktop application to verify that it works.

Effects:

Will provide more accessible real-time monitoring of vehicle health, with a lower barrier to entry compared to other products on the market.

Technical approach:

Python-obd to communicate with the vehicle, ELM327-emulator to emulate a vehicle, HTML, CSS, and JavaScript to build the user interface.

Risks:

Learning curve of GUI and OBD-2 libraries

Misjudging the complexity of tasks leading to time loss.

Possible incompatibilities with certain vehicles or other technologies.

Main Features:

Ability to show diagnostic codes while driving in an intuitive manner, with more serious codes being given priority.

A driving mode that only displays data to the user and a parked mode that will allow the user to configure the software.

Ability for the user to configure the data displayed based on available sensors while in park mode.

Ability to track diagnostics over time to compare and visualize

Save driving data to log

Stretch Goals:

Maintenance interval tracker that keeps track of standard maintenance items such as oil change intervals, and user-defined intervals for different maintenance items.

Develop app for mobile devices

Depending on limitations, contribute to python-OBD library

Include suggested fixes to problems

Use Cases:

Peyton's Use Case: Tracking data

Actors: User

1. Triggers: Tracker being turned on
2. Preconditions: The tracker is turned on
3. Postconditions: The data from the users drive is successfully logged
4. List of steps:
 - A user opens the app and turns on the tracker
 - The user then goes for a drive and parks at their destination
 - The user then filters the display to show the data they want to see
 - The user now sees their diagnostics from the drive with some visualizations
 - The user saves the data to their log
 - The user then goes to their log
 - The data from their recent drive is there
5. Extensions/variations of the success scenario: The user forgets to turn on the tracker
6. Exceptions: The data is incorrectly stored in log and lost

Owen's Case: Error Codes:

Actor: User

1. Trigger: ECU sets error code
2. Precondition: No current error codes in ECU
3. Postcondition: Error code is displayed to user
4. List of Steps:
 - App is on and connected
 - ECU sets an error code
 - Code is transmitted through OBD2 to the app
 - App reads code and determines severity
 - code is stored in app
 - if code is severe immediately alerts user
 - else code is displayed when user is stopped
5. Extensions:
 - If no codes are set the program will not alert the user
6. Exceptions:
 - ECU sets a code and it is never displayed to the user even when stopped

Silas' Use Case: Driving mode:

Actor: The User or the vehicle's velocity.

1. Trigger: If the vehicle starts moving (velocity exceeds 0-5mph), or the user selects driving mode.
2. Precondition: The app is not already in driving mode.

3. Postcondition: Driving mode is activated, simplifying the GUI so that the user can see a single graph and a few stats at a glance and see severe error code popups, but can't interact with the app at all, other than to leave driving mode if their speed is low enough.
4. List of Steps:
 - The app is running and the vehicle has been started.
 - The vehicle starts moving.
 - Driving mode is activated, and it switches interfaces.
 - The vehicle stops moving.
 - The user selects to exit driving mode, or the vehicle is turned off.
 - The app exits driving mode.
5. Extensions: The user can select to enter or exit driving mode while parked, and driving mode will automatically exit when the vehicle is turned off.
6. Exceptions / potential failures: Driving mode will rely mostly on the vehicle's velocity, so it will only work correctly as long as that data is accurate (which it really should be).

Tiernan's Use Case: Data Visualization and Tracking

Actors: User

1. Trigger: User will move to separate page that displays visualization, tracking will be kept in logs from driving history
2. Preconditions: There must be previous data to display, or currently reading data from OBD-2 scanner
3. Postconditions: User will see easily readable graph and statistics
4. List of steps:
 - User selects what to monitor and log
 - Data is generated or fetched
 - Data is rendered and displayed
 - Data is saved to log for historical tracking
5. Extensions: No data is stored, nothing to read. User doesn't save output
6. Exception: Data is unreadable for some reason. Too much data crashes

Non-Functional Requirements:

1. Response time and switching between pages should not be laggy, less than 0.5 seconds
2. Serious error codes should be displayed within 30-40 seconds after check engine light activates. Non-serious error codes should be displayed within 1 minute of vehicle exiting driving mode
3. Data log should be accessible to a user outside of the program

External Requirements:

- The product must be robust against errors that can reasonably be expected to occur, such as invalid user input.

- **How the product will be installable:** We can provide a demo version of the desktop app that does not require the emulator to run, and we will keep the full version available, but it will require complex driver installations on Windows for the emulator.
- **The software will be buildable from source by others,** and the instructions for how to install the drivers for the emulator will be in the GitHub repository.
- **The scope of the project is reasonable:** We all believe this is true.

Team Process Description:

Describe your quarter-long development process.

- **We will use python-OB2** because it is the simplest way to pull data from the OBD2 port. We will use HTML, CSS and JavaScript for the user interface because we are the most familiar with these languages and we feel will have the most flexibility in building our project. We will use GitHub Projects to help track and manage our tasks. We will use this because it will put all of the information for our project in one place, and will make it easier to manage tasks.
- **Schedule of milestones for each member for each week:**
 Github project tasks completed and ready for testing within each task's allotted time frame. Will be updated over time. Early tasks include defining the data schema, allowing the software to interact with the emulator, and figuring out the necessary tech stack
- **Three major risks:**
 1. Misjudging the complexity of certain tasks
 2. Not all vehicles may support the features we design around
 3. Front and back end data specifications could be mismatched, make sure the data schema is clearly defined early on
- **Describe at what point in your process external feedback (i.e., feedback from outside your project team, including the project manager) will be most useful and how you will get that feedback.**

External Feedback will be most useful in testing the user experience and ensuring it is accessible and easy to use by most users. We will gather this feedback by having classmates and friends try the software and provide feedback.

Timeline: <https://github.com/users/owenkoster/projects/2/views/4>

Software Architecture

Our software will use a layered architecture, since it must involve communication between several different levels. The user's car or the emulator will communicate with our OBD Communication module, which will communicate with a GUI module.

Interfaces:

- The OBD Communication module communicates through a USB (or simulated USB) to the car (or emulator), or via TCP to the emulator.
- The OBD Communication module will send the data from the car to the GUI Module using websockets.

Our software will mostly be working with live data, and data will only be saved as logs when the user requests it. Logs will be written to text files and read when the user requests to view one.

The main reason we chose a layered architecture is that we want to use a Python library for communicating with OBD-2 and a Java library for the GUI.

Pros/Cons of layered architecture we chose: Separating processes makes it easier to test and make changes to different code separately, but communication between processes will make development significantly harder.

Software Design

We will use Python-OBD to communicate with the vehicle. We will use Java Swing to build the user interface. We will use ZeroMQ to communicate between the Python script and the Java application.

The Java application will send data requests to the Python script, the Python script will respond with the requested data from a table that is updated every .5 seconds with data queried from the vehicle or emulator.

The Python script will write to JSON files for log data.

The Python script will read JSON log files and store the contained data in a separate set of tables for quick access.

Coding guideline

We chose this style guide because it is the standard provided by python:

<https://peps.python.org/pep-0008/>

We chose this style guide for Java because it is simple and intuitive:

<https://www.geeksforgeeks.org/java/coding-guidelines-in-java/>

Guidelines will be enforced by having a team member review code for violations and make changes as needed.

Risk Assessment

1. Not all vehicles may support the features we design around

The likelihood is low. The impact is high. We have already tested the emulator and a real vehicle, and the results show that the vehicle can be queried for available commands. By knowing which commands are available, we can dynamically adjust the commands the user can use. This problem will appear very quickly during testing, and we plan to build tests to verify that the user cannot send invalid commands to the car. To mitigate this, should it appear, we will add additional input filtering to prevent this.

2. Misjudging the complexity of certain tasks.

The likelihood of this occurring is medium, and the impact would be medium. We based this estimate upon prior projects and experience, where we tend to take on more than we can handle. The steps we are taking to prevent this are using languages familiar to team members and demonstrating that the library we plan to use to interact with cars and the emulator can do what we need. The issue will be detected when it occurs, and symptoms include missed deadlines and roadblocks. To mitigate it, we will constantly evaluate our progress and monitor issues.

3. Front and back end data specifications could be mismatched.

The likelihood of this occurring is low, and the impact would be high. We based this estimate on prior experience. The steps we are taking to prevent this are to define a unified data specification before we begin writing any code. This problem will be easy to detect and will appear during initial testing and possibly during development. If this issue occurs, we will update our data specification and fix the affected scripts to ensure compliance.

4. Overestimating the capabilities of our chosen languages.

The likelihood of this occurring is low, and the impact would be high. We based this estimate on research done into Python and Java, along with prior experience in these languages. The steps we are taking to prevent this are conducting research into the languages we use to ensure they fit our use case and can perform the required tasks. This problem will be detected during development when we cannot complete a feature due to limitations. To mitigate this, we will either use a different approach to the feature or, in a worst-case scenario, switch to a tool that better supports our use case.

5. Risk of sending bad commands to the car's computer.

Unlikely that library or program will send bad or potentially destructive instructions to the car's computer, but it is possible, and more likely that the user or developer could send bad commands that might cause an error in the car or wipe its memory. Impact could be medium or high. OBD scanners can send codes to clear engine codes, it is possible that we could accidentally clear an engine code with a wrong command. We plan to avoid that problem by only sending queries to the car to receive data, and not implement code clearing functionality. Detecting it should be simple by comparing the state of the ECU before and after using our program. Should it occur, disconnecting the car battery for a period of time does a hard reset on the car computer.

Project Schedule

To manage the project, we use GitHub Projects to track tasks. We chose this because it is a single location for our project management and the project itself. GitHub Project also enables us to set a timeline for each project, assign teammates to each task, and track the progress of each task. GitHub Projects also works seamlessly with GitHub issues. Here is a link to the project: <https://github.com/users/owenkoster/projects/2/views/1>. The project is organized so that higher-priority items are at the top of the timeline and lower-priority items are at the bottom. Items that must be completed before others are placed on the timeline with earlier completion dates, and the card will explicitly specify prerequisites. Teammates will assign themselves to any card they are actively working on and mark the card in the Board as in progress, and move it to in review when task is ready for testing, and it will be moved to done when the task has been tested or graded. No teammate will take more than 2 tasks at once to help ensure even task loading and allow other teammates to help share the load. Multiple teammates may work on the same task together, provided there is communication.

Team Structure

Owen: Project Management, Python backend. Project management is important because it enables us to keep track of tasks and progress. The Python backend is important because it links the car and the GUI. I feel I am suited for this role because I have training and experience in project management, and I also have experience working in Python and with the Python-OBD library.

Peyton: Python, Testing & Debugging, GUI. The GUI is important because the user needs a clean and easy way to interact with the software. Testing and debugging is important to make sure everything runs smoothly. I feel confident in my abilities to review and test code. I will also help out with Python and GUI as needed. I have some experience programming in Python. I also have experience working on GUI from a previous Web Development course.

Tiernan: Python scripts, Data management, environment emulation. For data to be usable, it must be saved in a clearly defined format that can be read by the user and also visualize data inside the application. Previous experience with python lends me better to these roles than for Java

Silas: GUI. I have experience working with java swing from my previous software engineering course, so I will be in charge of the GUI implementation. I will work to divide up tasks related to GUI implementation and help teammates with getting started.

Test plan & Bugs

Test the communication layer between the vehicle and our program. This can be done with mock data sent to our program from the emulator.

Test data logging. To test our data logging, we can send our log back into the program and see if it reads correctly. Can also compare logs to known test/mock data.

Test the communication layer between the Python script and Java Swing. This can be done by initially sending test strings through. We will then test whether the data received from Python is correct and whether the commands from Java are correct. We will then test bad commands and bad data. This step is important because it is a core component of the project and must be well tested.

Graphing. Test the graphing capabilities by sending known data to ensure it displays properly. Test with random or mock data to ensure that real-world data displays in a readable way.

We will use GitHub issues to track bugs and other issues as they occur.

We use GitHub actions to run automatic unit tests when code is pushed to GitHub.

Detailed Testing Process

Unit Tests:

Owen: test_CommandFiltration.py: Tests that the program correctly filters out unwanted commands. (passed)

Peyton: test_TripLogger.py: Tests when you log an entry, the CSV file is written correctly, specifically that the PID, Value, Unit, and DTC_Codes columns all contain the expected values. (passed)

Tiernan: Tested port filters to block ports like bluetooth and invalid system ports and return correct port (passed)

Validation Tests:

Owen: test_CacheCallback.py: Tests that the cache callback correctly updates the cache. (passed)

Peyton: test_ValidateTripLogging.py: Simulates a full driving session by logging 5 sensor readings, then validates the resulting CSV file. (passed)

Tiernan: Tested playback function to ensure logs are correctly read including timestamp and mimicking delay to simulate real data. (passed)

Integration Tests:

Owen: test_ZMQPublish.p: Simulates data messages being sent over the network port. The test checks that a valid message works and an invalid message does not. (Passed)

Silas: Manually checks that UI features all work, and the program finishes its execution correctly. (Failed, If the program is stopped while log playback is running, the program will run in the background until log playback is finished).

Peyton: test_IntegrationTrip.py Simulates the real data flow of the program: CacheCallback → TripLogger → PublishVehicleData (passed)

Tiernan: Manually tested UI and backend to ensure proper shutdown is being sent, processes terminated, and ports freed. (passed)

System Tests:

Owen: test_Playback.py. Simulates a full playback of 2 log files and asserts that the data was read in correctly. (passed)

Silas: Manually tests that the program runs successfully and does not hang when run differently: with emulator, with docker, with vehicle. (passed)

Peyton: test_SystemCrossPlatform.py: Simulates the program once on Windows and once on MacOS (passed)

Tiernan: Manually tested UI playback flow, verified import, UI transitions, and dynamically rendered content. (passed)

Documentation plan

The documentation will include installation instructions for the user to start using the program. Further guides will showcase usage and features of the application, and instruct the user how to use them. As we develop the app, we will add clear documentation strings for functions that will easily translate to an end-user guide.

Reflections:

Peyton: One lesson I learned is how to get different files and parts of the program to communicate with each other even if they are written in different languages using ZeroMQ. A second lesson I learned is how to navigate working as a team on GitHub using a KANBAN on GitHub projects. I haven't done anything like it before and found it very useful and very helpful in making sure everyone is on the same page about what we are all individually assigned to do, and where we are at on progress as a whole with the project. And a third lesson I have learned is how build files work with continuous integration and testing.

Side note: Not all of my contributions show up in the contributors tab on GitHub due to an issue with GitHub Desktop.

Silas: Throughout the development of this project, I learned quite a lot through annoying troubleshooting. I learned new things about maven, python, pip, java versions, intellij, GitHub Desktop, OneDrive, windows device manager, optional windows updates, JSON objects and JFreeChart. Everything else I did in java swing was straight forward, and the only thing I planned on learning was JFreeChart, but all of the other stuff I learned from having trouble getting everything to work, mostly related to the vehicle emulator.

Owen: One lesson I learned is how to use Docker to set up a container for the emulator, making it easier to test our code. I learned how CI works and how to set up a CI workflow with automated testing in GitHub Actions. I was able to test my knowledge of how project management works and how effective a KANBAN is at organizing tasks for a small team.

Tiernan: I learned a lot about GitHub and version control, which kinda scared me before in group projects. I learned a lot about terminal and command lines and how useful they can be. I learned a bit of java which I had no experience in before. I also learned about build systems and continuous integration. I learned about tradeoffs between different database formats like CSV over JSON, which could cause issues if not saved correctly or software crashes.